Rencana Pembagian Tugas dan Spesifikasi Teknis: MET di Microsoft Azure

Real-world cloud dengan algoritma heuristik Minimum Execution Time (MET)

Strategi Optimasi Komputasi Awan - lanjutan dari simulasi CloudSim | Revisi pembagian tugas dari Versi 4

Item	Keterangan
Tim	Kelompok terdiri atas 5 anggota. Anggota 1 adalah pemilik akun Azure for Students dan pembuat semua instance Azure.
Akun Azure	Milik Anggota 1, kredit dinyatakan masih penuh. Langkah pertama Selasa malam: memverifikasi angka kredit dan tanggal kedaluwarsanya.
Algoritma	MET (wajib, sama dengan tugas lama) dan Round Robin sebagai pembanding.
Workload	Pemrosesan gambar nyata dengan Pillow (blur, resize, sharpen, kompresi JPEG). Dosen hanya mensyaratkan algoritma MET, workload bebas.
Demo	Kamis, 29 Oktober 2026, 19.30 WIB, 15-30 menit: struktur, bukti sudah dijalankan, kesesuaian dengan algoritma, grafik hasil.
Keluaran	Kode 5 komponen + 2 skrip pendukung, grafik hasil, dan laporan akhir.
Dibuat	Selasa, 27 Oktober 2026. Revisi pembagian tugas dari Versi 4.

1. Tujuan dan Pemenuhan Syarat Dosen

Syarat / kemungkinan permintaan	Cara memenuhinya
Cloud nyata	Microsoft Azure (Azure for Students).
1 datacenter	1 region Azure, Resource Group rg-cloud-met, VNet vnet-datacenter (10.0.0.0/16).
Minimal 2 host	Host A = subnet-host-a, Host B = subnet-host-b. Azure tidak mengekspos server fisik, jadi subnet mewakili host (sama dengan kelompok lain). Konfirmasikan ke dosen.
Minimal 2 VM per host	Host A: vm-worker-1, vm-worker-2. Host B: vm-worker-3, vm-worker-4. Ditambah vm-master di subnet-mgmt.
Algoritma heuristik yang sama (MET)	scheduler.py memilih VM dengan ET(i,j) = panjang_task_i / kapasitas_j terkecil (logika sama dengan applyMET di repo lama).
Load balancing (jika diminta)	MET lalu Round Robin dengan data sama; bandingkan distribusi task, makespan, DI, utilisasi (ada grafik).
Ganti spesifikasi (jika diminta)	az vm resize satu worker, jalankan master.py lagi (kalibrasi otomatis di awal), bandingkan hasil sebelum dan sesudah.
Tes lain (jika diminta)	Failover: matikan satu worker; master melewatinya dan menjadwalkan ulang. Uji beban dengan jumlah gambar lebih besar.

2. Arsitektur Azure

Host / Subnet	VM	Ukuran	vCPU	IP privat tetap	Peran
Host A / subnet-host-a (10.0.1.0/24)	vm-worker-1	Standard_B2s	2	10.0.1.11	Worker
Host A	vm-worker-2	Standard_B1s	1	10.0.1.12	Worker
Host B / subnet-host-b (10.0.2.0/24)	vm-worker-3	Standard_B2s	2	10.0.2.11	Worker
Host B	vm-worker-4	Standard_B1s	1	10.0.2.12	Worker
subnet-mgmt (10.0.3.0/24)	vm-master	Standard_B1s	1	10.0.3.10	Menjalankan master.py dan menyimpan dataset; satu-satunya VM ber-IP publik

- Total 7 vCPU. Jika kuota hanya 6 vCPU, ubah vm-worker-3 menjadi Standard_B1s (satu variabel di deploy.sh).
- Worker tanpa IP publik, diakses lewat vm-master (jump host). SSH hanya dengan kunci. Lalu lintas port 8000 antar-VM sudah diizinkan aturan bawaan VNet Azure.
- Disk murah: --storage-sku StandardSSD_LRS. VM seri B memakai kredit CPU: jangan benchmark besar Kamis siang, nyalakan VM sekitar 17.30.
- VM B1s hanya memiliki RAM sekitar 1 GB: ukuran gambar maksimum 2560x1440 dan hanya sedikit; pemrosesan satu gambar per vCPU.
- Hemat kredit: VM menyala hanya saat dipakai (./deploy.sh stop dan start).

2.1 Cara Mengakses Azure dan Memindahkan File

Azure tidak hanya bisa dipakai lewat SSH. Ada tiga tempat bekerja: laptop/Cloud Shell (mengelola Azure), Portal web (melihat status), dan terminal SSH (masuk ke dalam VM Linux).

Aktivitas	Dilakukan di	Caranya
Membuat, menghapus, stop, start, resize VM	Laptop Anggota 1 (bash/WSL/Git Bash dengan Azure CLI) atau Azure Cloud Shell di browser	deploy.sh (perintah az). deploy.sh tidak dijalankan di dalam VM.
Memasang paket dan layanan di VM	Tidak perlu SSH	az vm run-command invoke, dikirim dari az CLI.
Memindahkan kode ke VM	Dari repo GitHub kelompok	Di VM: git clone (pertama) atau git pull. Alternatif: scp. Copy-paste manual hanya untuk perbaikan kecil lewat nano.
Menjalankan master.py dan dataset_generator.py	Terminal SSH ke vm-master	ssh azureuser@IP_publik_master
Debug worker	SSH lewat master (jump host)	ssh -J azureuser@IP_publik_master azureuser@10.0.1.11
Melihat status, kredit, ukuran VM	Portal web (portal.azure.com)	Klik; juga dipakai untuk menunjukkan arsitektur saat demo.
Grafik dan laporan	Laptop	./deploy.sh pull menyalin folder results dari master, lalu python plot_results.py.

Semua VM dapat mengakses internet keluar secara bawaan, sehingga git clone dan pip install juga bisa dari worker tanpa IP publik. Jika repo pribadi, pakai personal access token atau jadikan repo publik selama tugas berlangsung.

3. Akun dan Kredit Azure

- Anggota 1 adalah pemilik akun dan pembuat seluruh instance.
- Langkah pertama (Selasa malam, sebelum membuat apa pun): buka Cost Management, Credits, catat sisa kredit dan tanggal kedaluwarsa.
- Kebutuhan kredit kira-kira USD 3-8 untuk seluruh pengerjaan jika VM dimatikan saat tidak dipakai (perkiraan kasar, cek Azure Pricing Calculator).
- Setelah verifikasi: hapus resource lama yang masih menagih, pasang budget alert, cek region yang diizinkan dan kuota vCPU (az vm list-usage).

4. Pembagian Tugas dan Keseimbangan Beban

Tujuh keluaran program dan satu laporan dibagi ke lima orang. Perkiraan waktu kerja inti sekitar 4-5 jam per orang dengan bantuan AI. Urutan kerja: verifikasi akun dan kredit, menyiapkan dataset dan worker lokal, menyelesaikan scheduler dan master, membuat infrastruktur dan deploy ke Azure, pengujian dan bukti, grafik, lalu laporan.

Anggota	Komponen utama	Bagian laporan	Est.
1	deploy.sh (semua subcommand: create, stop, start, delete, push, pull); verifikasi kredit Azure, budget alert, kuota dan region. Pemilik akun Azure dan pembuat semua instance.	Bab 1: Pendahuluan dan Arsitektur Azure	5 jam
2	dataset_generator.py, worker.py, run_local.sh, dan workers.local.json.	Bab 2: Dataset dan Worker	5 jam
3	scheduler.py (MET, Round Robin, metrik, explain_met, selftest).	Bab 3: Landasan Algoritma dan Penjadwal	4 jam
4	master.py (kalibrasi, dispatch, pencatatan CSV).	Bab 4: Master dan Alur Eksekusi	4,5 jam
5	plot_results.py, pengujian gabungan T0-T8, README.md, folder bukti, penyusunan dan penggabungan laporan akhir.	Bab 5: Pengujian, Hasil, Analisis, dan Kesimpulan; menggabungkan seluruh laporan	5 jam

Urutan ketergantungan: Anggota 1 memverifikasi kredit, budget, region, dan kuota sebelum membuat infrastruktur. Anggota 2 menyiapkan dataset_generator.py, worker.py, run_local.sh, dan workers.local.json agar semua anggota dapat menguji secara lokal dengan 4 worker localhost. Anggota 3 (pemilik scheduler.py) dan Anggota 4 (pemilik master.py) hanya terhubung lewat fungsi met, rr, dan compute_metrics; explain_met tetap tersedia sebagai fungsi diagnostik pada scheduler.py sesuai spesifikasi teknis. Setelah komponen lokal terintegrasi, Anggota 5 membuat plot_results.py dengan data contoh dari format CSV pada 6.2, mengoordinasikan pengujian gabungan, menyimpan bukti, dan menggabungkan laporan. Perubahan kontrak pada Bagian 6 harus diberitahukan ke seluruh tim.

5. Jadwal

Waktu	Kegiatan (sesuai urutan Anggota 1-5)	Penanggung jawab	Gerbang
Selasa 27 Okt malam	1) Anggota 1 memverifikasi kredit, tanggal kedaluwarsa, region, kuota, dan budget alert. 2) Anggota 2 menyelesaikan dataset, worker, dan konfigurasi 4 worker localhost. 3) Anggota 3 mulai scheduler.py dan menyiapkan selftest. 4) Anggota 4 menyiapkan master.py untuk integrasi lokal. 5) Anggota 5 membaca kontrak, menyiapkan struktur bukti dan README. Semua anggota membaca Bagian 6 dan memakai prompt sesuai komponen.	Anggota 1, 2, 3, 4, 5	GATE 1: kredit cukup untuk 3-8 USD?
Rabu 28 Okt pagi	1) Anggota 1 membuat infrastruktur (create) setelah kredit terverifikasi, lalu menghentikan VM jika belum dipakai. 2) Anggota 2 memastikan 4 worker localhost berjalan. 3) Anggota 3 menyelesaikan scheduler.py dan selftest. 4) Anggota 4 menyelesaikan integrasi master.py; uji MET dan RR end-to-end secara lokal. 5) Anggota 5 mulai plot_results.py dengan data contoh CSV.	Anggota 1, 2, 3, 4, 5	GATE 2: MET dan RR berjalan end-to-end di lokal.
Rabu siang-sore (VM menyala 5-6 jam)	1) Anggota 1 menjalankan ./deploy.sh push dan mengelola VM. 2) Anggota 2 memastikan kode worker dan dataset siap pada lingkungan target. 3) Anggota 3 memvalidasi perilaku MET/RR dan metrik. 4) Anggota 4 menjalankan master.py di Azure untuk kalibrasi, dispatch, dan pencatatan CSV. 5) Anggota 5 mengoordinasikan T4-T8, menyimpan bukti, menjalankan ./deploy.sh pull, lalu plot_results.py. Lakukan uji beban, resize, dan failover. Setelah itu stop VM dan bekukan kode.	Anggota 1, 2, 3, 4, 5	GATE 3: MET dan RR sukses di Azure dan grafik terbentuk.
Rabu malam	1) Anggota 1 menulis bagian Bab 1. 2) Anggota 2 menulis bagian Bab 2. 3) Anggota 3 menulis bagian Bab 3. 4) Anggota 4 menulis bagian Bab 4. 5) Anggota 5 menulis bagian Bab 5, memeriksa bukti dan grafik. Setiap bagian sekitar satu halaman dan memakai hasil aktual.	Anggota 1, 2, 3, 4, 5	Semua bagian terkumpul.
Kamis 29 Okt siang	1) Anggota 1 memeriksa ulang informasi arsitektur dan biaya. 2) Anggota 2 merapikan bagian dataset dan worker. 3) Anggota 3 merapikan bagian algoritma. 4) Anggota 4 merapikan bagian master dan alur eksekusi. 5) Anggota 5 menyusun serta menggabungkan laporan akhir, merapikan README, dan mengatur latihan penjelasan tiap komponen. VM mati.	Anggota 1, 2, 3, 4, 5	Laporan versi akhir.
Kamis 17.30-18.45	1) Anggota 1 menjalankan ./deploy.sh start. 2) Anggota 2 memastikan endpoint worker menjawab. 3) Anggota 3 memeriksa scheduler dan hasil MET. 4) Anggota 4 menjalankan satu run MET singkat. 5) Anggota 5 memeriksa folder bukti, grafik, dan laporan serta memimpin latihan 15 menit.	Anggota 1, 2, 3, 4, 5	Checklist Bagian 9.
Kamis 29 Okt, 19.30 WIB	DEMO: tunjukkan arsitektur, struktur kode, bukti pengujian, grafik hasil, dan kesesuaian dengan MET. Setelah demo jalankan ./deploy.sh delete.	Anggota 1, 2, 3, 4, 5	Demo selesai; resource dihapus.

6. Spesifikasi untuk AI (bagian yang diberikan ke AI)

Cara pakai: tiap anggota menempelkan 6.1 Konteks Proyek, 6.2 Kontrak Data, lalu spesifikasi komponen yang dikerjakan (6.3-6.9) ke AI bersama template di 6.10. Hasil kode wajib berjalan dengan perintah uji yang tertera. Jangan menambah fitur di luar spesifikasi.

6.1 Konteks Proyek

Kami kelompok 5 orang, mata kuliah Strategi Optimasi Komputasi Awan. Tugas dosen: membangun cloud nyata (bukan simulasi) di Microsoft Azure: 1 datacenter, 2 host, 2 VM per host, lalu menjalankan algoritma heuristik MET (Minimum Execution Time) yang dahulu kami implementasikan di CloudSim (repo https://github.com/PlasmaOneCode/SimulasiMET). Workload bebas; kami memakai pemrosesan gambar nyata dengan Pillow (blur, resize, sharpen, kompresi JPEG) sebagai task.

Arsitektur: 1 region, 1 VNet. Host A = subnet-host-a (vm-worker-1, vm-worker-2). Host B = subnet-host-b (vm-worker-3, vm-worker-4). vm-master di subnet-mgmt. Tiap worker menjalankan worker.py (FastAPI, port 8000). dataset_generator.py membuat gambar sintetis dan manifest.csv. master.py membaca manifest, mengukur kapasitas tiap worker (kalibrasi), meminta scheduler.py memilih VM (MET atau

Round Robin), mengunggah tiap gambar via HTTP ke worker terpilih, menghitung metrik, dan menyimpan CSV. plot_results.py membuat grafik dari CSV.

Teknologi: Python 3.10+, FastAPI, uvicorn, requests, Pillow, numpy, matplotlib, pandas. Tanpa Docker, tanpa database.

Batasan: waktu 2 hari dan kredit Azure terbatas. Kode harus SEDERHANA: satu file per komponen, komentar singkat, mudah dijalankan, tanpa fitur tambahan. Demo: Kamis 29 Okt 2026, 19.30 WIB; kemungkinan diminta tes load balancing, ganti spesifikasi VM, dan failover.

Struktur repo: worker.py scheduler.py master.py dataset_generator.py plot_results.py deploy.sh run_local.sh workers.json workers.local.json; data/images/ data/manifest.csv; results/summary.csv results/tasks_*.csv results/figures/*.png results/hasil_ringkasan.md.

6.2 Kontrak Data (tidak boleh diubah sepihak)

- length sebuah task = width x height / 10000 (satuan abstrak sebanding dengan beban). Kapasitas VM = satuan length per detik saat semua vCPU dipakai (hasil kalibrasi). Estimasi ET(i,j) = length_i / capacity_j.
- vm_id sama persis dengan nama VM (vm-worker-1 ... vm-worker-4) dan dengan kolom vm_id di workers.json.
- Satuan waktu: detik (float). Waktu hanya diukur dengan jam master (time.perf_counter), kecuali exec_s yang diukur di worker.

```
GET /health -> {"status":"ok","vm_id":"vm-worker-1","cores":2}
POST /process?task_id=7&target_scale=0.5
body: byte gambar (Content-Type application/octet-stream)
-> {"task_id":7,"vm_id":"vm-worker-1","exec_s":0.84,"out_bytes":153200,"ok":true}

data/manifest.csv: task_id,filename,width,height,size_bytes,length
workers.json: [{"vm_id":"vm-worker-1","host":"A","url":"http://10.0.1.11:8000"}, ...]
records (list of dict, satu per task selesai):
{"task_id":7,"filename":"img_0007.jpg","length":207.36,"vm_id":"vm-worker-1",
 "send_s":0.02,"finish_s":0.9,"exec_s":0.84}
results/tasks_<algo>_<timestamp>.csv: task_id,filename,length,vm_id,send_s,finish_s,exec_s
results/summary.csv (satu baris per run; kolom *_per_vm berisi string JSON):
timestamp,label,algo,n_tasks,makespan_s,avg_wait_s,throughput,DI,avg_util_pct,
tasks_per_vm,util_per_vm,capacity_per_vm
```

6.3 deploy.sh (Anggota 1) - satu skrip dengan subcommand

Jalankan di laptop atau Azure Cloud Shell (bukan di dalam VM):

```
./deploy.sh create | stop | start | delete | push | pull
```

Variabel di atas file: RG=rg-cloud-met, LOC=southeastasia, ukuran tiap VM, IP statis tiap VM.

create (set -e; berhenti saat error):

- az group create; VNet vnet-datacenter 10.0.0.0/16; subnet subnet-host-a 10.0.1.0/24, subnet-host-b 10.0.2.0/24, subnet-mgmt 10.0.3.0/24; NSG nsg-datacenter (izinkan 22 saja).
- 4 worker (tanpa IP publik: --public-ip-address "") dan 1 master (IP publik), semua Ubuntu2204, --storage-sku StandardSSD_LRS, --private-ip-address sesuai tabel arsitektur.
- Di tiap VM pasang python3-venv dan paket via az vm run-command invoke --command-id RunShellScript. Worker: fastapi uvicorn pillow numpy. Master: requests pillow numpy. Worker dibuat otomatis hidup setelah reboot (systemd unit atau cron @reboot) dengan perintah uvicorn worker:app --host 0.0.0.0 --port 8000.

```
stop    : az vm deallocate semua VM (hemat kredit).
start   : az vm start semua VM.
delete  : az group delete --yes --no-wait.
push    : memperbarui kode di semua VM. Cara utama: repo GitHub kelompok; di tiap VM jalankan
          git clone (pertama kali) atau git pull lewat az vm run-command invoke, lalu restart
          layanan worker.
          Alternatif tanpa GitHub: scp -J azureuser@<IP publik master> ke tiap VM.
pull    : scp folder results/ dari master ke ./results lokal (untuk plot_results.py).
```

Cetak IP publik master dan ringkasan akhir. Perintah manual terpisah (tidak dalam skrip):

```
az vm resize -g rg-cloud-met -n vm-worker-1 --size Standard_B1s
```

Uji: dari vm-master jalankan curl http://10.0.1.11:8000/health (dan 3 worker lain) -> semua ok.

Catatan: sebelum create jalankan az vm list-usage -l southeastasia -o table untuk cek kuota. Anggota 1 juga memverifikasi kredit Azure, tanggal kedaluwarsa, budget alert, dan region sebelum membuat semua instance.

6.4 dataset_generator.py (Anggota 2)

Tujuan: membuat gambar sintetis deterministik dan manifest.csv.

```
python dataset_generator.py --count 100 --out data/images --seed 42
```

- Komposisi ukuran (acak berseed): 30% 640x480, 30% 1280x720, 30% 1920x1080, 10% 2560x1440.
- Isi gambar: gradien warna + beberapa bentuk acak + noise (numpy), agar kompresi nyata.
- Simpan JPEG quality 92 sebagai img_0001.jpg dst; tulis data/manifest.csv (kolom pada 6.2) dengan length = width*height/10000 dan size_bytes dari file.
- Cetak ringkasan: jumlah per ukuran, total byte, total length.

Uji: jalankan dua kali dengan seed sama -> manifest.csv identik. Semua gambar dapat dibuka Pillow.

6.5 worker.py (Anggota 2)

Tujuan: layanan yang memproses gambar sungguhan di tiap VM.

```
uvicorn worker:app --host 0.0.0.0 --port 8000
```

- Startup: ProcessPoolExecutor(max_workers=CORES); CORES = env WORKER_CORES jika ada, jika tidak os.cpu_count(). VM_ID = env VM_ID jika ada, jika tidak hostname. Konstanta PASSES = 2 (jumlah ulangan pipeline, untuk menyetel beban per gambar).
- GET /health: status ok, vm_id, cores.
- POST /process?task_id=<int>&target_scale=<float, default 0.5>: body = byte gambar. Baca dengan await request.body(); body kosong -> HTTP 400. Jalankan process_image di proses anak lewat loop.run_in_executor(pool, ...) (bukan thread, GIL). Respons pada 6.2.

process_image(data, scale, passes) -> (out_bytes, ok, exec_s), fungsi tingkat atas modul:

```
t = perf_counter(); ulangi passes kali:
 Image.open(BytesIO(data)).convert("RGB") ->
 GaussianBlur(radius=2) -> resize((w*scale, h*scale), Image.Resampling.LANCZOS) ->
 UnsharpMask -> save ke BytesIO (JPEG, quality=85, optimize=True).
ok = hasil dapat dibuka ulang dan lolos Image.verify().
exec_s = perf_counter() - t.
```

- Tidak menyimpan file ke disk. Tanpa autentikasi (hanya VNet atau localhost).

Uji:

```
WORKER_CORES=2 VM_ID=vm-worker-1 uvicorn worker:app --port 8001
curl localhost:8001/health
curl -X POST "localhost:8001/process?task_id=1" -H 'Content-Type: application/octet-stream' \
   --data-binary @data/images/img_0001.jpg  # exec_s lebih dari 0, ok true
```

Uji paralelisme: kirim 2 request bersamaan ke worker 2 core; waktu total mendekati 1x exec_s. Penyetelan: PASSES agar gambar 1920x1080 sekitar 0,3-0,8 detik pada satu core B1s.

6.6 run_local.sh dan workers.local.json (Anggota 2)

Tujuan: seluruh tim dapat menguji tanpa Azure.

- run_local.sh menjalankan 4 worker: port 8001..8004 dengan VM_ID=vm-worker-1..4 dan WORKER_CORES=2,1,2,1 (meniru ukuran B2s,B1s,B2s,B1s).
- Proses latar belakang; satu perintah untuk menghentikan semuanya (trap atau file pid).
- workers.local.json memuat 4 entri dengan URL http://127.0.0.1:800x; host A untuk worker 1-2, host B untuk worker 3-4.

6.7 scheduler.py (Anggota 3) - fungsi murni, tanpa jaringan

Tujuan: algoritma MET (utama), Round Robin (pembanding), dan metrik. Hanya pustaka standar.

```
def met(lengths: list[float], capacity: dict[str, float], healthy: list[str],
        tol: float = 0.05) -> list[str]
   Untuk tiap task i: ET_j = lengths[i] / capacity[j] untuk j di healthy (urutan dipertahankan).
   best = min ET_j. Pilih VM pertama (urutan healthy) dengan ET_j <= best * (1 + tol).
   Abaikan beban VM (sifat dasar MET). Kembalikan list vm_id per task.

def rr(n: int, healthy: list[str]) -> list[str]  # task i -> healthy[i % len(healthy)]

def compute_metrics(records: list[dict], cores: dict[str, int], makespan: float) -> dict
   makespan_s = makespan; avg_wait_s = rata-rata send_s; throughput = n / makespan
   T_j = finish_s terbesar pada VM j (0 jika VM tidak menerima task); semua VM sehat dihitung
   DI = (max T_j - min T_j) / rata-rata T_j (jika semua beban di 1 dari 4 VM, DI = 4)
   util_per_vm[j] = sum(exec_s di j) / (cores[j] * makespan) * 100; avg_util_pct = rata-ratanya
   tasks_per_vm = dict vm_id -> jumlah task

def explain_met(lengths, capacity, healthy, n=3) -> str
   Tabel teks: untuk n task pertama tampilkan ET ke setiap VM dan VM terpilih (bukti MET).
```

Selftest: python scheduler.py --selftest.

- 20 VM, kapasitas 1000/1500/2000/2500 (5 tiap tipe), 1000 task acak: semua task harus jatuh ke satu VM tercepat; DI model muatan (T_j = total length di j / kapasitas_j) harus 20.0, sama dengan hasil repo SimulasiMET. Cetak PASS atau FAIL.
- Uji tambahan: dua VM berkapasitas sama dalam toleransi 5% -> VM pertama menang; VM tercepat dikeluarkan dari healthy -> task pindah ke tercepat berikutnya.

6.8 master.py (Anggota 4) - satu skrip CLI

```
python master.py --config workers.json --manifest data/manifest.csv \
  --images data/images --algo MET --limit 100 [--label baseline] [--out results]
```

--algo: MET | RR | BOTH (BOTH = MET lalu RR, data sama). --label: teks bebas (mis. baseline, setelah_resize, failover) yang dicatat di summary.csv. Impor: from scheduler import met, rr, ... Kontrak fungsi utama integrasi dengan scheduler.py: met, rr, dan compute_metrics; explain_met dipakai untuk diagnostik sesuai spesifikasi scheduler.

Langkah:

1. Baca manifest; ambil --limit baris pertama. Jangan memuat semua gambar ke memori: baca file dari disk saat akan dikirim (RAM master hanya sekitar 1 GB).
2. GET /health tiap worker (timeout 3 s). Tidak menjawab = tidak sehat; cetak status semua VM.

3. Kalibrasi: buat satu gambar kalibrasi 1920x1080 di memori (length 207.36). Ke tiap worker sehat kirim paralel cores request /process berisi gambar itu; T = waktu dinding sampai semua selesai; capacity = cores * 207.36 / T. Cetak kapasitas tiap VM.
4. Jadwal: panggil met() atau rr(); cetak explain_met() untuk 3 task pertama.
5. Estimasi: cetak sum(length) / max(capacity) detik; beri peringatan jika di luar 30-180 detik (sesuaikan --limit atau PASSES pada worker).
6. Eksekusi: t0 = perf_counter(). Per worker satu ThreadPoolExecutor(max_workers=cores), tiap thread memanggil POST /process (blocking) untuk task milik worker itu sesuai urutan. Catat send_s dan finish_s relatif t0 dari jam master, exec_s dari respons. Jika request gagal: tandai VM tidak sehat dan jadwalkan ulang task itu ke VM sehat lain dengan algoritma sama.
7. compute_metrics(...); cetak tabel per VM (jumlah task, total exec, util) dan metrik.
8. Simpan results/summary.csv (append, header jika file baru) dan results/tasks_<algo>_<ts>.csv.

Uji lokal:

```
python master.py --config workers.local.json --manifest data/manifest.csv \
 --images data/images --algo BOTH --limit 20
```

Harapan: MET menaruh hampir semua task di worker berkapasitas tertinggi (DI mendekati 4); RR membagi rata (jumlah task per VM selisih maksimal 1); semua respons ok true.

6.9 plot_results.py (Anggota 5)

Jalankan: python plot_results.py --results results (membaca summary.csv dan tasks_*.csv).

Keluaran di results/figures/ (matplotlib, judul dan label sumbu berbahasa Indonesia, dpi 150):

No.	Nama file	Isi
1	makespan_comparison.png	Bar makespan MET vs RR (kelompokkan menurut label dan n_tasks).
2	task_distribution.png	Jumlah task per VM, MET vs RR berdampingan.
3	imbalance_di.png	Bar Degree of Imbalance MET vs RR.
4	utilization_per_vm.png	util_per_vm (persen) tiap VM untuk MET dan RR.
5	throughput.png	Throughput MET vs RR.
6	gantt_<algo>.png	Garis waktu tiap VM (send_s sampai finish_s tiap task) dari run terakhir tiap algoritma.
7	before_after_resize.png	Bandingkan run berlabel baseline dan setelah_resize (capacity_per_vm, makespan, VM tujuan MET); lewati jika label tidak ada.

Juga tulis results/hasil_ringkasan.md: tabel Markdown semua baris summary.csv. Jika sebuah grafik tidak punya data, lewati dengan pesan, jangan error.

Uji: buat summary.csv contoh 4 baris (MET dan RR, 2 label) -> semua grafik terbentuk. Data contoh harus mengikuti format CSV pada 6.2.

6.10 Template Prompt untuk AI

```
[tempel 6.1 Konteks Proyek dan 6.2 Kontrak Data]
Saya Anggota <N> dan bertugas menulis <nama file>. Spesifikasinya:
[tempel spesifikasi file dari 6.3-6.9]
Permintaan: tulis satu file lengkap yang langsung berjalan. Ikuti nama, parameter, format JSON dan
CSV persis seperti spesifikasi. Jangan menambah fitur atau file lain. Setelah kode, berikan:
(1) perintah menjalankan dan menguji, (2) daftar cek hasil yang saya harus lihat, dan (3) hal
yang mungkin salah. Jika ada bagian spesifikasi yang ambigu, tulis asumsi Anda di awal jawaban.
```

6.11 Pengujian Gabungan dan Bukti (Anggota 5)

No.	Uji	Hasil yang diharapkan
T0	python dataset_generator.py --count 100	100 gambar dan manifest.csv; dua kali jalan dengan seed sama menghasilkan manifest sama.
T1	python scheduler.py --selftest	PASS, DI model = 20.0.
T2	run_local.sh lalu master.py --algo BOTH --limit 20	MET: sebagian besar task di VM kapasitas tertinggi; RR: merata; semua ok true; CSV tertulis.
T3	Matikan satu worker lokal, ulangi T2	Master melewati worker itu dan tetap menyelesaikan semua task.
T4	Di Azure: master.py --algo BOTH --limit 100 --label baseline	Makespan MET lebih besar dari RR; DI MET mendekati 4.
T5	Di Azure: --limit 300 (uji beban)	Throughput dan utilisasi tercatat; tidak ada kegagalan memori di B1s.
T6	az vm resize vm-worker-1 ke B1s; --algo MET --label setelah_resize	Kapasitas hasil kalibrasi berubah dan pilihan MET pindah ke VM tercepat berikutnya.
T7	Stop satu VM (deallocate); --label failover	VM dilewati, tidak ada task gagal.
T8	plot_results.py pada data hasil T4-T7	Grafik 1-7 dan hasil_ringkasan.md terbentuk.

Bukti disimpan di satu folder: CSV, grafik, tangkapan layar portal Azure dan terminal untuk tiap uji. Anggota 5 mengoordinasikan pengumpulan dan memastikan semua bukti tersedia untuk laporan dan demo.

6.12 Laporan Akhir

Anggota 5 menyusun dan menggabungkan seluruh laporan. Setiap anggota menulis bab sesuai tanggung jawab berikut.

Bab	Isi	Penulis
Bab 1. Pendahuluan dan Arsitektur Azure	Latar belakang, tujuan, syarat dosen, ruang lingkup; diagram datacenter, host, VM, jaringan; ukuran VM; biaya; prosedur resize dan stop/start.	Anggota 1
Bab 2. Dataset dan Worker	Pembuatan dataset, manifest, pipeline pemrosesan gambar dengan Pillow, layanan worker dan endpoint.	Anggota 2
Bab 3. Landasan Algoritma dan Penjadwal	Algoritma MET dan Round Robin, rumus ET, metrik (makespan, waiting time, throughput, DI, utilisasi), serta implementasi scheduler.	Anggota 3
Bab 4. Master dan Alur Eksekusi	Kalibrasi, pemeriksaan kesehatan worker, dispatch task, failover, pencatatan CSV, dan alur data.	Anggota 4
Bab 5. Pengujian, Hasil, Analisis, dan Kesimpulan	Skenario T0-T8, tabel hasil_ringkasan, grafik 1-7, perbandingan MET vs RR, penyebab DI tinggi, pengaruh resize dan failover, perbandingan dengan simulasi CloudSim, kesimpulan dan kendala.	Anggota 5; juga menyusun dan menggabungkan seluruh laporan

7. Alur Demo (target 15-20 menit)

Menit	Isi	Pelaksana
0-3	Arsitektur: 1 datacenter, 2 host, 2 VM per host + master; tunjukkan portal Azure.	Anggota 1
3-8	Struktur kode; pipeline gambar di worker, fungsi MET dan tabel keputusan ET(i,j), serta alur master.	Anggota 2, 3, 4
8-13	Bukti sudah dijalankan: grafik MET vs RR (makespan, distribusi task, DI, utilisasi); run singkat live jika VM menyala.	Anggota 3, 5
13-18	Hanya bila diminta: uji beban 300 gambar, resize live (mulai awal, butuh 1-3 menit), failover; tampilkan hasil Rabu sebagai pembanding.	Anggota 1, 5
18-20	Kesimpulan: MET menumpuk beban di VM tercepat (DI = 4 dari 4 VM, sama dengan DI = 20 di CloudSim).	Anggota 3, 5

8. Risiko dan Mitigasi

Risiko	Mitigasi
Kredit akun kurang atau kedaluwarsa	Verifikasi Selasa malam; VM dimatikan saat tidak dipakai; budget alert.
Region dibatasi / kuota vCPU kecil	Pakai region yang diizinkan; vm-worker-3 menjadi B1s (total 6 vCPU).
Kredit CPU seri B habis, atau RAM B1s penuh	Uji pendek; kalibrasi otomatis tiap run; gambar maksimum 2560x1440 dan sedikit; satu gambar per vCPU.
Hasil MET tampak buruk	Karakter MET (sama dengan DI = 20 di CloudSim), bukan bug.
Kode dari AI tidak cocok antar-komponen	Kontrak 6.2 mengikat; T0-T3 dijalankan lokal sebelum Azure.
Waktu tidak cukup	Prioritas: (1) MET dan RR di Azure, (2) grafik 1-4, (3) resize, (4) failover, (5) grafik 5-7.

9. Checklist Go/No-Go (Kamis 18.45)

- Kredit tersisa dicatat Anggota 1; koneksi internet kedua (hotspot) siap.
- Folder bukti, grafik, dan laporan tersedia offline di laptop Anggota 5.
- ./deploy.sh start selesai; 4 worker menjawab /health; satu run MET 100 gambar berhasil dalam 2 menit terakhir.